Table des matières

Cours 08 — Les codes ..... 1
Étape 1 — Une cible pour la CI, et son entrepôt jetable · Fiabiliser et faire tourner ..... 1
Étape 2 — Le linteur · Fiabiliser et faire tourner ..... 2
Étape 3 — La fraîcheur de la caisse · STAGING ..... 4
Étape 4 — Le contrat du fait des ventes · MARTS ..... 5
Étape 5 — La CI, sur une copie neuve du dépôt · Fiabiliser et faire tourner ..... 8

# Cours 08 — Les codes

*Cours 8/9 · La Brûlerie — Formation Analytics Engineering*

| STAGING | INTERMEDIATE | MARTS |
|---|---|---|
| reprendre la `RAW`, une vue par table | joindre, clarifier, préparer l'étoile | concevoir et livrer les étoiles |
| ✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue | ✓ | ✓ |

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin. Un fichier qui existait déjà se montre en entier, dans son nouvel état, ceux que le linteur a corrigés compris. La structure des sources, `02_ae/ci/structure_sources.sql`, n'est pas ici : la commande de l'étape 1.2 l'écrit.

## Étape 1 — Une cible pour la CI, et son entrepôt jetable · Fiabiliser et faire tourner

### 02_ae/ci/profiles.yml

```yaml
# 02_ae/ci/profiles.yml : la connexion de la CI, à un entrepôt jetable ; versionnée, rien n'y est secret
brulerie:
  target: ci
  outputs:
    ci:
      # 1. le moteur, et l'entrepôt jetable, vu depuis le dossier d'où l'on lance dbt
      type: duckdb
      path: plateforme/entrepot/ci.duckdb
```



© 2026 iAlexMG. Tous droits réservés.

### 02_ae/.gitignore

```
# Ce que dbt refait à chaque commande : Git ne le garde pas
target/
logs/
# L'identifiant que dbt écrit à côté du profil de la CI : il est propre au poste
.user.yml
```

## Étape 2 — Le linteur · Fiabiliser et faire tourner

### 02_ae/.sqlfluff

```
# 02_ae/.sqlfluff : la forme du SQL du projet, que le linteur fait respecter
[sqlfluff]
# 1. le moteur, et le templater qui compile le Jinja de dbt avant de lire
dialect = duckdb
templater = dbt
# 2. la longueur d'une ligne de code, puis les règles écartées, avec leur raison :
# ST06 et ST09 réordonneraient les colonnes et les jointures sans rien corriger ;
# LT08 et LT09 déplieraient les tests courts au-delà de 20 lignes
max_line_length = 100
exclude_rules = ST06, ST09, LT08, LT09

[sqlfluff:templater:dbt]
# 3. le projet, et le profil de la CI, vus depuis ce fichier
project_dir = .
profiles_dir = ci
target = ci

[sqlfluff:indentation]
# 4. le contenu d'un bloc de Jinja garde le retrait du SQL qui l'entoure
template_blocks_indent = false

[sqlfluff:rules:capitalisation.keywords]
# 5. les mots-clés en minuscules, comme tout le SQL du projet
capitalisation_policy = lower

[sqlfluff:rules:layout.long_lines]
# 6. un commentaire peut dépasser : la première ligne de chaque fichier dit ce qu'il fait
ignore_comment_lines = true
```



© 2026 iAlexMG. Tous droits réservés.

## 02_ae/models/marts/dim_date.sql

```sql
-- 02_ae/models/marts/dim_date.sql : les jours du 1er mai 2023 au 30 juin 2025, un par ligne
select
    -- 1. la clé : la date elle-même
    jour::date as date_jour,
    -- 2. le nom du jour, du lundi (1) au dimanche (7)
    case isodow(jour)
        when 1 then 'lundi' when 2 then 'mardi' when 3 then 'mercredi'
        when 4 then 'jeudi' when 5 then 'vendredi' when 6 then 'samedi'
        when 7 then 'dimanche'
    end as nom_jour,
    -- 3. la fin de semaine : le samedi et le dimanche
    isodow(jour) >= 6 as est_fin_de_semaine
-- 4. une ligne par jour, de l'ouverture du programme de fidélité au 30 juin, les deux bornes comprises
from generate_series(date '2023-05-01', date '2025-06-30', interval 1 day) as jours (jour)
```

## 02_ae/snapshots/snp_prix.sql

```sql
{# 02_ae/snapshots/snp_prix.sql : le prix de chaque produit, une version par prix, tirée du journal #}
{% snapshot snp_prix %}
{{ config(
    unique_key='produit_id', strategy='check', check_cols=['prix'], updated_at='en_vigueur_le'
) }}
select
    {# 1. le produit, son prix, et le jour où ce prix est entré en vigueur #}
    produit_id::integer as produit_id,
    nouvelle_valeur::decimal(10, 2) as prix,
    date_trunc('day', horodatage::timestamp) as en_vigueur_le
from {{ source('brulerie', 'raw_modifications') }}
{# 2. le catalogue au jour regardé : les créations et les changements de prix connus ce jour-là #}
where
    champ in ('CREATION', 'PRIX')
    and horodatage::date <= '{{ var("jour", "2025-06-30") }}'
{# 3. de chaque produit, le dernier prix connu #}
qualify row_number() over (partition by produit_id order by horodatage desc) = 1
{% endsnapshot %}
```

## 02_ae/models/marts/fct_ventes.sql

```sql
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
```



© 2026 iAlexMG. Tous droits réservés.

```
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait, et l'instant de l'export qui l'a livrée
    ticket_id,
    ligne,
    exporte_le,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
-- 4. une passe ne garde que les lignes d'un export plus récent que le dernier déjà rangé
{% if is_incremental() %}
where exporte_le > (select max(f.exporte_le) from {{ this }} as f)
{% endif %}
```

## Étape 3 — La fraîcheur de la caisse · STAGING

### 02_ae/models/staging/_sources.yml

```
# 02_ae/models/staging/_sources.yml : la RAW reçue, déclarée une seule fois
sources:
  - name: brulerie
    description: "La RAW du café, tout en texte : chaque export gardé, même deux fois."
    # 1. les tables de la RAW vivent dans le schéma par défaut de DuckDB
    schema: main
    # 2. une entrée par table qu'un modèle lit
    tables:
      - name: raw_tickets
        description: "Les tickets de la caisse, un par ligne et par export."
        # 3. la caisse exporte chaque soir : sa fraîcheur se lit à l'instant de son dernier export
        config:
          loaded_at_field: "strptime(replace(split_part(filename, '_', 3), '.csv', ''), '%Y%m%d%H%M')"
          freshness:
            warn_after: {count: 1, period: day}
            error_after: {count: 2, period: day}
      - name: raw_lignes
        description: "Les lignes des tickets, un produit vendu par ligne et par export."
```



© 2026 iAlexMG. Tous droits réservés.

```
      config:
        loaded_at_field: "strptime(replace(split_part(filename, '_', 3), '.csv', ''), '%Y%m%d%H%M')"
        freshness:
          warn_after: {count: 1, period: day}
          error_after: {count: 2, period: day}
    - name: raw_produits
      description: "Le catalogue de la caisse au 2025-06-30."
    - name: raw_membres
      description: "Les membres du programme de fidélité au 2025-06-30."
    - name: raw_modifications
      description: "Le journal des modifications du catalogue, une ligne par valeur changée, depuis 2022."
    - name: raw_lots
      description: "Le journal des lots du torréfacteur, un lot par ligne, en juin 2025."
    - name: raw_consentements
      description: "Le journal des consentements des membres, une ligne par décision, depuis 2023."
```

## Étape 4 — Le contrat du fait des ventes · MARTS

### 02_ae/models/marts/_marts.yml

```
# 02_ae/models/marts/_marts.yml : les dimensions et les trois faits, décrits et testés
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les jours du 1er mai 2023 au 30 juin 2025, un par ligne, fabriqués sans source."
    columns:
      - name: date_jour
        description: "La clé : la date elle-même."
        data_tests:
          - unique
          - not_null
  - name: dim_produit
    description: "Un produit du catalogue par ligne, au 2025-06-30."
    columns:
      - name: cle_produit
        description: "La clé de substitution, md5 du numéro du produit."
        data_tests:
          - unique
          - not_null
      # 2. une liste fermée, et son not_null à côté
      - name: categorie
```



© 2026 iAlexMG. Tous droits réservés.

```yaml
      data_tests:
        - not_null
        - accepted_values:
            arguments:
              values: ['ACCESSOIRE', 'BOISSON_CHAUDE', 'BOISSON_FROIDE', 'GRAINS', 'SANDWICH', 'VIENNOISERIE']
- name: dim_membre
  description: "Un membre du programme de fidélité par ligne, sans rien de personnel."
  columns:
    - name: cle_membre
      description: "La clé de substitution, md5 du numéro de la carte."
      data_tests:
        - unique
        - not_null
# 3. le fait, bâti par passes et tenu par son contrat : chaque colonne typée, chaque clé dans sa dimension
- name: fct_ventes
  description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
  config:
    materialized: incremental
    incremental_strategy: delete+insert
    unique_key: [ticket_id, ligne]
    on_schema_change: fail
    contract:
      enforced: true
  columns:
    - name: cle_date
      data_type: date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_type: varchar
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_produit')
              field: cle_produit
    - name: cle_membre
      description: "Vide pour un ticket sans carte de membre."
```



© 2026 iAlexMG. Tous droits réservés.

```yaml
      data_type: varchar
      data_tests:
        - relationships:
            arguments:
              to: ref('dim_membre')
              field: cle_membre
    - name: ticket_id
      data_type: integer
    - name: ligne
      data_type: integer
    - name: exporte_le
      data_type: timestamp
    - name: quantite
      data_type: integer
    - name: montant_brut
      data_type: decimal(18, 2)
    - name: rabais
      data_type: decimal(10, 2)
    - name: montant
      data_type: decimal(10, 2)
# 4. le fait d'accumulation : une ligne par lot, et chaque clé dans sa dimension
- name: fct_lots
  description: "Le fait d'accumulation des lots du torréfacteur, une ligne par lot, un jalon par colonne."
  columns:
    - name: lot_id
      description: "Le numéro du lot : le grain du fait."
      data_tests:
        - unique
        - not_null
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_produit')
```



© 2026 iAlexMG. Tous droits réservés.

```
              field: cle_produit
  # 5. le fait sans mesure : chaque clé dans sa dimension, et le statut dans une liste fermée
  - name: fct_consentements
    description: "Le fait sans mesure des consentements, une ligne par décision d'un membre."
    columns:
      - name: cle_date
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_date')
                field: date_jour
      - name: cle_membre
        data_tests:
          - not_null
          - relationships:
              arguments:
                to: ref('dim_membre')
                field: cle_membre
      - name: statut
        description: "Ce que dit la décision : accordé, refusé ou retiré."
        data_tests:
          - not_null
          - accepted_values:
              arguments:
                values: ['ACCORDE', 'REFUSE', 'RETIRE']
```

## Étape 5 — La CI, sur une copie neuve du dépôt · Fiabiliser et faire tourner

### 02_ae/ci/verifier.py

```python
# 02_ae/ci/verifier.py : la CI locale, qui vérifie le dernier commit sur une copie neuve du dépôt
import os
import subprocess
import sys
import tempfile
from pathlib import Path

# 1. le dépôt, où que le script soit lancé ; dbt cherchera le projet dans 02_ae, comme chez vous
DEPOT = Path(__file__).resolve().parents[1]
os.environ["DBT_PROJECT_DIR"] = "02_ae"
```



© 2026 iAlexMG. Tous droits réservés.

```
# 2. les étapes, dans l'ordre : la copie, l'entrepôt jetable, la forme, puis le fond
ETAPES = [
    f'git clone --quiet "{DEPOT}" 02_ae',
    "duckdb plateforme/entrepot/ci.duckdb -f 02_ae/ci/structure_sources.sql",
    "sqlfluff lint 02_ae/models 02_ae/tests 02_ae/snapshots --config 02_ae/.sqlfluff",
    "dbt build --target ci --profiles-dir 02_ae/ci",
]
# 3. un dossier neuf, qui s'efface à la fin, avec le dossier de l'entrepôt jetable
with tempfile.TemporaryDirectory() as dossier:
    (Path(dossier) / "plateforme" / "entrepot").mkdir(parents=True)
    for etape in ETAPES:
        print(f"\n> {etape}", flush=True)
        # 4. la CI s'arrête à la première étape en échec, avec un code d'erreur
        if subprocess.run(etape, cwd=dossier, shell=True).returncode != 0:
            sys.exit(f"\nCI en échec : {etape}")
print("\nCI au vert : le dernier commit passe la forme et le fond, sur une copie neuve.")
```



© 2026 iAlexMG. Tous droits réservés.